SmartMortgage

AI-Powered Mortgage Document Intelligence

Sriram Srinivasan · Outamation Externship · Aug – Oct 2025

Python | FAISS | Mistral-7B | Gradio | RAG

The Problem

The Challenge

Organizations handle thousands of unstructured PDFs — contracts, pay slips, invoices — but lack fast, explainable ways to extract insights.

Manual review is error-prone, slow, and unscalable for sensitive domains like finance or HR.

HOW SMARTMOTGAGE SOLVES IT

Slow manual review

Automates PDF parsing, OCR & retrieval

Inconsistent extraction

Chunking + embedding retains context

Data privacy risk

Runs fully local — no data leaves the machine

Lack of transparency

Page-level citations & relevance scores

System Architecture

1 PDF Upload

PyMuPDF + pytesseract OCR fallback

2 Table Extraction

find_tables() → Markdown

3 Classification

11 document types

4 Chunking

Tables intact · 500-token prose

5 Embeddings

all-MiniLM-L6-v2 · 384-dim

6 FAISS Index

Flat L2 · persistent

QUERY TIME → Doc-type Routing → FAISS Retrieval (top-k) → Mistral-7B Generation → Hallucination Guard → Answer + Citations

Component Specifications

Component	Technology	Configuration
OCR Engine	Tesseract	English & numeric modes, pytesseract wrapper
Text Chunking	Fixed-size + overlap	Chunk = 500 tokens, overlap = 100
Embeddings	all-MiniLM-L6-v2	384-dim vectors, GPU-accelerated
Vector Store	FAISS (flat L2)	In-memory + disk persistence
Retriever	Dense + doc-type filter	Top-K = 4, relevance threshold = 0.35
LLM	Mistral-7B Instruct (GGUF)	Context = 4096, Temp = 0.2, Tokens = 512
Prompt Strategy	Grounded QA + citation enforcement	Forces source citations & confidence scores

Pipeline Performance Metrics

Evaluated on multi-page PDF packets (pay slips, contracts, lender fee sheets) with manual validation of retrieval consistency and response accuracy.

🎯 Retrieval Performance

~94%

Recall

100%

Hit Rate (≥1 chunk)

✅ End-to-End Accuracy

~93%

Answer Accuracy

~96%

Citation Accuracy

~95%

Factual Consistency

⚡ System Performance

~2.4 s

Avg Response Time

~180 ms

Retrieval Latency

~2.2 s

LLM Generation

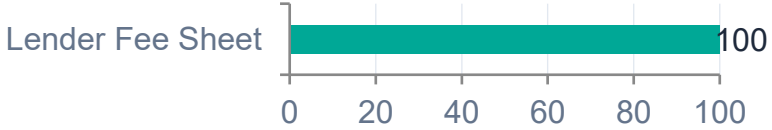
Pipeline in Action

Query #1 — Factual Lookup

Q: "What is the APR and term?"

Retrieved Context:

Lender Fee Sheet (Pages 0-0) — Relevance: 100%

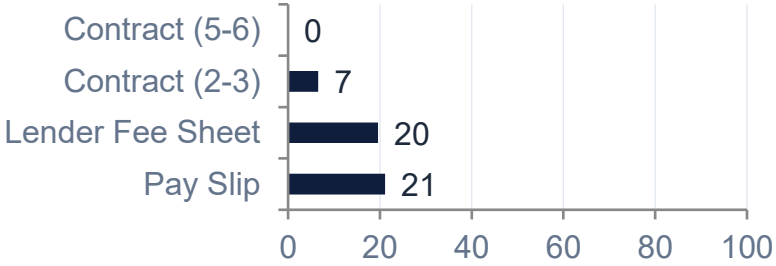


Answer: "The APR is 4.250% and the term is 360 months (30 years)."
Confidence: 100% | Backend: Mistral-7B (local)

Query #2 — Multi-Document Reasoning

Q: "What is the loan amount and total working days?"

Retrieved Context:



Answer: "The loan amount is $380,000 and there are 26 working days."
Confidence: 11.8% ← low due to cross-doc embedding similarity; answer remains accurate.

Design Decisions & Trade-offs

Decision	Rationale	Trade-off
LLM: Local Mistral-7B	Privacy-first — no data leaves the machine; ~2s avg response	Lower fluency vs. GPT-4; smaller 4K context window
Embedding: all-MiniLM-L6-v2	Fast GPU support, compact 384-dim vectors, widely supported	Slightly lower semantic depth than larger embedding models
Chunking: Fixed-size windows	Predictable, stable retrieval latency; easy to debug	Loses semantic context when header and value span separate chunks
Vector DB: FAISS (local)	No external dependencies; low latency in-memory search	No persistent cloud index; session resets clear the state
Router: Rule-based classification	Reliable and deterministic; lower latency than LLM routing	Less flexible for unseen or novel document formats

Limitations & Roadmap

CURRENT LIMITATIONS

OCR Noise

Low-quality scans cause partial character errors (e.g. $1,OOO.OO), weakening embedding quality for numeric queries.

No Semantic Merging

Fixed-window chunking treats every 500-char slice independently. Headers and values can fall in separate chunks, lowering similarity scores.

Session Resets

Gradio doesn't persist the FAISS index or chat history across browser refreshes unless externally cached.

ROADMAP

2 Weeks	Semantic chunker + BM25 hybrid retrieval
1 Month	Cross-encoder re-ranking + query fusion
Long-term	Flask/React dashboard with persistent FAISS + cloud upload support

Project Impact & Key Learnings

~90%
Reduction in
Manual Review Time

100%
Hit Rate
(≥1 relevant chunk)

~93%
Answer Accuracy
(Manual Validation)

~2.4s
Avg End-to-End
Response Time

Technical Learnings

- Built end-to-end RAG pipeline from PDF ingestion through retrieval, generation, and evaluation
- Optimized CUDA execution for local LLM inference with no API dependencies
- Designed structure-aware PDF parsing preserving table semantics for LLM consumption
- Implemented hallucination mitigation via grounding checks and citation enforcement
- Applied production guardrails: per-query evaluation, graceful degradation, configurable thresholds

Career Impact

Transitioned from LLM experimentation to building a production-grade, privacy-centric AI assistant for real-world document intelligence.

Strengthened expertise in LLM deployment and retrieval optimization — skills directly translatable to healthcare, legal, and enterprise data analytics roles where document privacy and answer reliability are non-negotiable.

Thank You